Phase A Report:

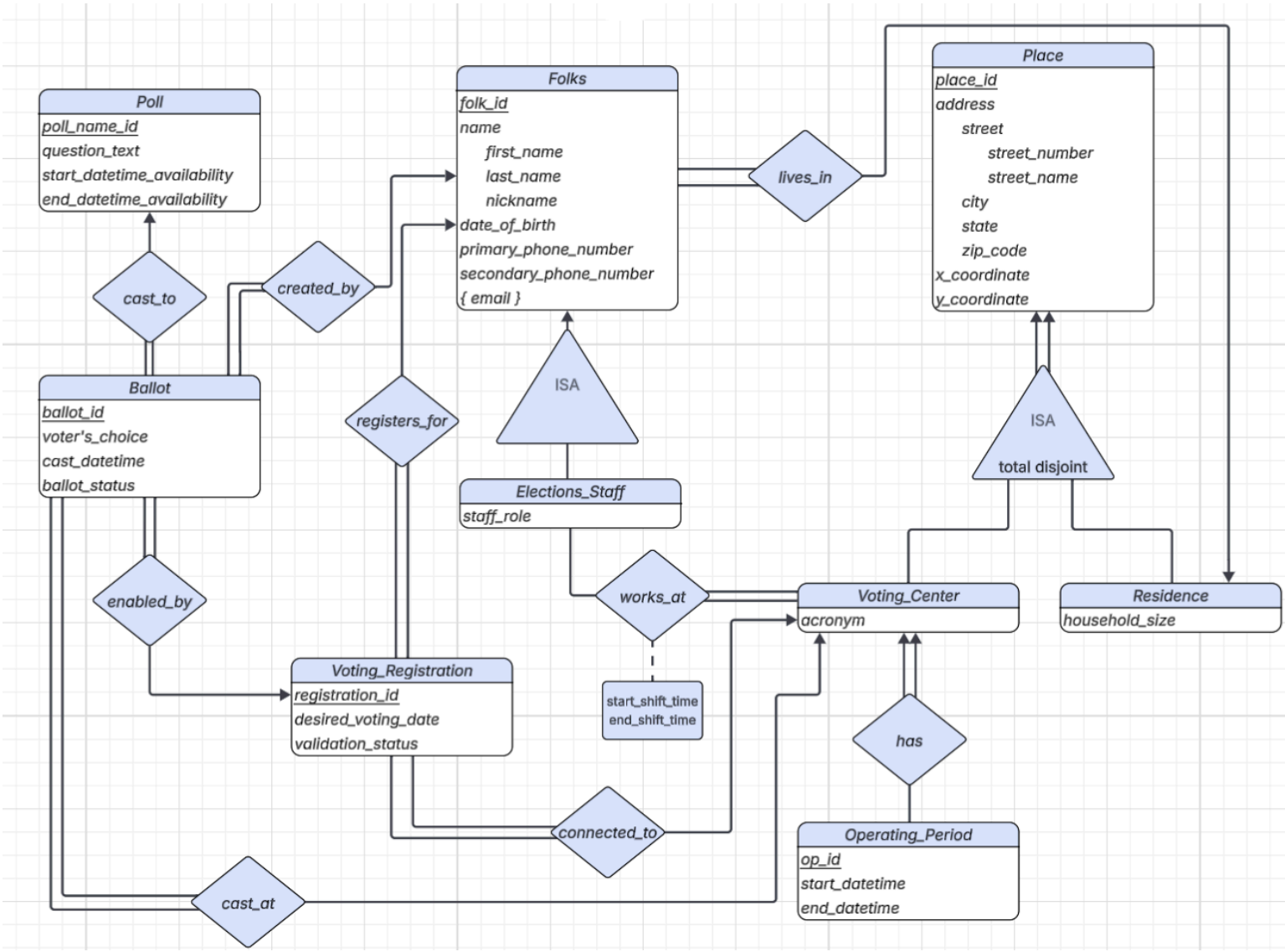
Instructions:

Executive summary that includes a brief introduction to the project, high level description of the requirements, and a succinct discussion of your accomplishments (as well as any significant limitations).

This project that I created is the Wonderland Elections Database Project, and the goal of the project was to design and implement a database application for the Elections Department of Wonderland. This database application will be used to manage the voting registrations, polls, and ballots cast by folks throughout Wonderland. The database application is meant to be used by the folks and elections staff of Wonderland. It will help folks(voters) register to vote, create ballots, and submit their votes for the poll. It will also help elections staff create polls, manage registrations, and count votes for ballots. The database also handles the information for each folk in Wonderland, the different residences and voting centers in Wonderland, and the operating periods for each voting center. The deliverables include MySQL files to create the database schema, delete the database schema, and fill the database with data for each relation. This MySQL database will be used by the Jupyter Notebook application, which will act as the user interface for the folks(voters) and elections staff to do a plethora of tasks such as creating a poll, creating a ballot, deleting a folk, and querying the database for various reports. Regarding some of the high-level requirements for the project, they are derived from the descriptions on the project document. The MySQL database maintains complex relationships between the relations Folks, Folks_Email, Elections_Staff, Place, Voting_Center, Residence, Operating_Period, Works_At, Voting_Registration, Ballot, and Poll. These relations include attributes of the information that is needed for the database, which is described in the project document. Some other requirements include maintaining the data integrity between these relations. The database enforces strict constraints and the relationships between these tables using primary keys, foreign keys, and attribute unique constraints. Also, the database imposes the correct data types for each attribute for each relation, and applies various logical constraints through triggers. Moreover, I implemented a serializable transaction for ballot insertions to prevent double-voting and ensure ACID compliance. Lastly, the user interface in Jupyter Notebooks uses Python functions and SQL Magic to interact with the database. The user interface fulfills all of the activities and reports described in the project document. Regarding my accomplishments throughout the creation of this project, the first part that I successfully created was the Entity-Relationship diagram for Phase B. To complete this, I used a diagram-creation app called LucidChart to create the visualization for each entity and their relationships with other entities. I was able to discern the entities needed, the attributes needed for each entity, and the relationships between the entities through the data requirements in the project document. I also created primary keys for each entity. Next, I reduced the E-R diagram I created to a relational schema. I mapped each entity to a relation based on the attributes and their type(simple, composite, multi-valued), and I mapped the relationships based on the cardinality, with many-to-many needing its own table, and all others being done through foreign keys. The creation of the relational schema leads to my next accomplishment, creating the SQL tables for the MySQL database. I was able to use CREATE TABLE to make the tables based on the relational schema, with the attributes being assigned the appropriate data types. I also created triggers to enforce various constraints described in the project document. Moreover, I created a SQL file to clear the database, and I defined another file to load sample data for each relation into the database. Lastly, my last accomplishment was making the user interface application for the database through Jupyter Notebooks. I was able to connect to the database using a configuration connection, and I defined Python functions with SQL queries, and separate SQL Magic queries to fulfill each activity and report defined in the project document. The application successfully gets user inputs and sends them to the database via Python functions. Moreover, I also created a transaction.sql file to implement the logic for ACID-compliant ballot insertions, and I made a queryAll.sql file to view the reports. Furthermore, I

created indices on the Place and Voting_Registration tables to optimize some queries in the report section. Finally, a limitation of my project is that the user interface is just a Jupyter Notebook environment. In the real-world, this would need an intuitive front-end to be usable for people. Moreover, another limitation is that my project does not include any security measures. All sensitive data can be queried using simple SQL statements. To be implemented in the real-world, this application would need security to protect people's data. Lastly, another limitation is that distance is calculated using the Pythagorean Theorem for my queries. However, in the real-world, an algorithm like Dijkstra's should be used to calculate the distance between two locations.

Phase B Report:



Entity Sets and Their Attributes

Entity: *Folks*
Primary Key: *folk_id*
Attributes:

- *folk_id*
- *name* (composite attribute)

        - *first_name*
        - *last_name*
        - *nickname*
    - *date_of_birth*
    - *primary_phone_number*
    - *secondary_phone_number*
    - *email* (multivalue attribute)

Entity: *Elections_Staff* (Specialization of *Folks* via ISA)
Primary Key: *folk_id* (inherited from *Folks* via ISA)
Attributes:
- *folk_id* (inherited from *Folks*)
- *staff_role*

Entity: *Place*
Primary Key: *place_id*
Attributes:
- *address* (composite attribute)
    - *street* (composite attribute)
        - *street_number*
        - *street_name*
    - *city*
    - *state*
    - *zip_code*
- *x_coordinate*
- *y_coordinate*

Entity: *Voting_Center* (Specialization of *Place* via ISA)
Primary Key: *place_id* (inherited from *Place* via ISA)
Attributes:
- *place_id* (inherited from *Place*)
- *acronym*

Entity: *Residence* (Specialization of *Place* via ISA)
Primary Key: *place_id* (inherited from *Place* via ISA)
Attributes:
- *place_id* (inherited from *Place*)
- *household_size*

Entity: *Operating_Period*
Primary Key: *op_id*
Attributes:
- *op_id*
- *start_datetime*

- *end_datetime*

Entity: *Poll*
Primary Key: *poll_name_id*
Attributes:
- *poll_name_id*
- *question_text*
- *start_datetime_availability*
- *end_datetime_availability*

Entity: *Ballot*
Primary Key: *ballot_id*
Attributes:
- *ballot_id*
- *voter's_choice*
- *cast_datetime*
- *validation_status*

Entity: *Voting_Registration*
Primary Key: *registration_id*
Attributes:
- *registration_id*
- *desired_voting_time*
- *validation_status*

Relationship Sets and Their Attributes

Relationship: *lives_in*
Entities Involved: *Folks* and *Residence*
Cardinality: Many-to-One
Participation: Total participation from *Folks* and partial participation from *Residence*
What this relationship means: All *Folks* reside at a *Residence*, and a *Residence* can hold multiple *Folks*.

Relationship: *works_at*
Entities Involved: *Elections_Staff* and *Voting_Center*
Cardinality: Many-to-Many
Participation: Partial participation from *Elections_Staff* and total participation from *Voting_Center*
Attributes:
- *start_shift_time*
- *end_shift_time*

What this relationship means: An elections staff member may work at many voting centers, and a voting center has many staff members. Total participation from *Voting_Center* means that every center must have staff members, but partial participation from *Elections_Staff* means that some staff are scheduled to work at a voting center and some are not scheduled at all. The attributes *start_shift_time* and *end_shift_time* represents the schedule the staff member is supposed to work.

Relationship: *has*
Entities Involved: *Voting_Center* and *Operating_Period*
Cardinality: One-To-Many
Participation: Total participation from *Voting_Center* and partial participation from *Operating_Period*
What this relationship means: A voting center must have one or more operating periods, hence total participation. Also, each operating period is unique to a voting center. Several operating periods may have the same *start_datetime* and *end_datetime*, but each is unique via the *op_id* primary key.

Relationship: *connected_to*
Entities Involved: *Voting_Registration* and *Voting_Center*
Cardinality: Many-To-One
Participation: Total participation from *Voting_Registration* and partial participation from *Voting_Center*
What this relationship means: Each voting registration is linked to one voting center, and one voting center has many registrations associated with it. Total participation from *Voting_Registration* means that each registration must be connected with a specific voting center.

Relationship: *registers_for*
Entities Involved: *Folks* and *Voting_Registration*
Cardinality: One-To-Many
Participation: Total participation from *Voting_Registration* and partial participation from *Folks*
What this relationship means: A Folk can have multiple valid registrations and total participation from *Voting_Registration* means that each registration must be associated with a Folk.

Relationship: *enabled_by*
Entities Involved: *Ballot* and *Voting_Registration*
Cardinality: Many-to-One
Participation: Total participation from *Ballot* and partial participation from *Voting_Registration*
What this relationship means: There are many different polls, hence many different ballots that need to be validated with one voting registration from a folk. Also, one registration yields ballots for multiple polls, thus connecting to many different ballots. Total participation from *Ballot* means that each ballot must be associated with a registration. However, not every registration produces a ballot because some voters may change their mind and choose to not vote, or vote in specific polls but not all.

Relationship: *cast_to*
Entities Involved: *Ballot* and *Poll*
Cardinality: Many-to-One
Participation: Total participation from *Ballot* and partial participation from *Poll*
What this relationship means: A poll can have many ballots and each ballot is cast to one specific poll. Total participation from *Ballot* means that every ballot belongs to a poll, but a poll can exist without ballots.

Relationship: *created_by*
Entities Involved: *Ballot* and *Folks*
Cardinality: One-to-One
Participation: Total participation from *Ballot* and partial participation from *Folks*

What this relationship means: One ballot for a specific poll is created by one folk. Total participation from *Ballot* means that every ballot belongs to a folk, but not every folk has to create a ballot.

Relationship: *cast_at*
Entities Involved: *Ballot* and *Voting_Center*
Cardinality: Many-to-One
Participation: Total participation from *Ballot* and partial participation from *Voting_Center*
What this relationship means: Every single ballot must be associated with one voting center, which is the center it was cast at, and each voting center can have many ballots cast there. *Ballot* has total participation because every ballot needs a voting center, but a voting center can exist without ballots, which is why it has partial participation.

Entity Sets and Their Constraints

Entity: *Folks*
Primary Key Constraints:
- The primary-key in the *Folks* entity is *folk_id*; this attribute must be unique and cannot be null

Unique Constraints:
- The attribute *primary_phone_number* must be unique
- The attribute *email* must be unique

Structural Constraints of Involved Relationships:
- Relationship of *Folks-lives_in-Place* is many-to-one
- Relationship of *Folks-registers_for-Voting_Registration* is many-to-one

Specialization Constraints:
- The *Folks* entity has a completion constraint of partial to the lower-level entity *Elections_Staff*

Domain Constraints:
- The value of the primary-key *folk_id* must be exactly 16 digits
- The value of the attributes *first_name*, *last_name*, and *nickname* must be alphabetical characters; cannot include numbers and special characters except for a dash
- The value of the attributes *primary_phone_number* and *secondary_phone_number* must be exactly 10 digit numbers; cannot include letters and special characters
- The value of the attribute *email* must contain an @

Not-Null Constraints:
- *first_name* attribute cannot be null
- *last_name* attribute cannot be null
- *primary_phone_number* attribute cannot be null
- *email* attribute cannot be null

Entity: *Elections_Staff* (Specialization of *Folks* via ISA)
Primary Key Constraints:
- The primary-key in the *Elections_Staff* entity is *folk_id*; this attribute is inherited from the *Folks* superclass entity and must be unique and not-null

Structural Constraints of Involved Relationships:

- Relationship of *Elections_Staff -works_at-Voting_Center* is many-to-many

Specialization Constraints:

- The entity *Elections_Staff* is a specialization of *Folks*; *Elections_Staff* will inherit all of the attributes of *Folks*

Domain Constraints:

- The value of the attribute *staff_role* must be either "clerk" or "monitor"

Not-Null Constraints:

- *staff_role* attribute cannot be null

Entity: *Place*

Primary Key Constraints:

- The primary-key in the *Place* entity is *place_id*; this attribute must be unique and cannot be null

Unique Constraints:

- All of the following attributes must be unique for each place: *address*, *x_coordinate* and *y_coordinate* pair

Structural Constraints of Involved Relationships:

- Relationship of *Folks-lives_in-Place* is many-to-one

Specialization Constraints:

- *Place* is involved in a total disjoint specialization, where every place is either a *Residence* or *Voting_Center*, but not both

Domain Constraints:

- The value of the attribute *place_id* must be alphanumeric and 10 characters
- The value of the attribute *street_number* must be an integer
- The value of the attribute *zip_code* must be an integer
- The value of the attributes *street_name*, *city*, and *state* must be alphabetical characters; cannot include numbers and special characters except for a dashes and spaces
- The value of the attributes *x_coordinate* and *y_coordinate* must be floats and fall within the valid ranges

Not-Null Constraints:

- None of the attributes in this entity are allowed to be null

Entity: *Voting_Center* (Specialization of *Place* via ISA)

Primary Key Constraints:

- The primary-key in the *Voting_Center* entity is *place_id*; this attribute is inherited from the *Place* superclass entity and must be unique and not-null

Unique Constraints:

- The attribute *acronym* must be unique

Structural Constraints of Involved Relationships:

- Relationship of *Elections_Staff -works_at-Voting_Center* is many-to-many
- Relationship of *Voting_Center-has-Operating_Period* is one-to-many
- Relationship of *Ballot-cast_at-Voting_Center* is many-to-one

Specialization Constraints:

- The entity *Voting_Center* is a specialization of *Place*; *Voting_Center* will inherit all of the attributes of *Place*

Domain Constraints:

- The value of the attribute *acronym* must be exactly 4 alphanumeric characters

Not-Null Constraints:

- The attribute *acronym* cannot be null

Other Constraints:

- A center's operating period can be modified only if nobody has registered to vote at the center during the associated time period

Entity: *Residence* (Specialization of *Place* via ISA)

Primary Key Constraints:

- The primary-key in the *Residence* entity is *place_id*; this attribute is inherited from the *Place* superclass entity and must be unique and not-null

Specialization Constraints:

- The entity *Residence* is a specialization of *Place*; *Residence* will inherit all of the attributes of *Place*

Domain Constraints:

- The value of the attribute *household_size* must be an integer

Not-Null Constraints:

- The attribute *household_size* cannot be null

Entity: *Operating_Period*

Primary Key Constraints:

- The primary-key in the *Operating_Period* entity is *op_id*; this attribute must be unique and cannot be null

Structural Constraints of Involved Relationships:

- Relationship of *Voting_Center-has-Operating_Period* is one-to-many

Domain Constraints:

- The attributes *start_datetime* and *end_datetime* must have datetime values
- The value of the attribute *start_datetime* must be an earlier date than the value of *end_datetime*

Not-Null Constraints:

- The attributes *start_datetime* and *end_datetime* must not be null

Other Constraints:

- An operating period can be modified only if nobody has registered to vote at the associated center during the time period

Entity: *Poll*

Primary Key Constraints:

- The primary-key in the *Poll* entity is *poll_name_id*; this attribute must be unique and cannot be null

Structural Constraints of Involved Relationships:

- Relationship of *Ballot-cast_to-Poll* is many-to-one

Domain Constraints:

- The value of the attribute *poll_name_id* must be exactly 4 alphanumeric characters
- The attributes *start_datetime_availability* and *end_datetime_availability* must have datetime values

- The value of the attribute *start_datetime_availability* must be an earlier date than the value of *end_datetime_availability*

Not-Null Constraints:

- None of the attributes in this entity are allowed to be null

Other Constraints:

- A poll cannot be modified if ballots exist for it
- A poll can only be deleted if it has no registrations

Entity: *Ballot*

Primary Key Constraints:

- The primary-key in the *Ballot* entity is *ballot_id*; this attribute must be unique and cannot be null

Structural Constraints of Involved Relationships:

- Relationship of *Ballot-cast_to-Poll* is many-to-one
- Relationship of *Ballot-enabled_by-Voting_Registration* is many-to-one
- Relationship of *Ballot-cast_at-Voting_Center* is many-to-one

Domain Constraints:

- The attribute *cast_datetime* must be a datetime value
- The value of the *voter's_choice* attribute must be "YES", "NO", "ABSTAIN", or "NULL"
- The value of the *validation_status* attribute must be "VALID" or "INVALID"

Not-Null Constraints:

- The attribute *validation_status* must not be null

Other Constraints:

- The attribute *cast_datetime* must be within the Voting_Center's Operating_Period

Entity: *Voting_Registration*

Primary Key Constraints:

- The primary-key in the *Voting_Registration* entity is *registration_id*; this attribute must be unique and cannot be null

Structural Constraints of Involved Relationships:

- Relationship of *Ballot-enabled_by-Voting_Registration* is many-to-one
- Relationship of *Voting_Registration-connected_to-Voting_Center* is many-to-one
- Relationship of *Folks-registers_for-Voting_Registration* is many-to-one

Domain Constraints:

- The value of the attribute *desired_voting_date* must be a date within a valid *Operating_Period*
- The value of the *validation_status* attribute must be "VALID" or "INVALID"

Not-Null Constraints:

- The attribute *desired_voting_date* must not be null

Other Constraints:

- A registration is valid only if the date is within the Operating_Period of the chosen Voting_Center

Relationship Sets and Their Constraints

Relationship: *lives_in*
Structural Constraints (Cardinality and Participation Constraints):

- Relationship of *Folks-lives_in-Residence* is many-to-one
- Total participation from *Folks* and partial participation from *Residence*

Relationship: *works_at*
Structural Constraints:

- Relationship of *Elections_Staff-works_at-Voting_Center* is many-to-many
- Partial participation from *Elections_Staff* and total participation from *Voting_Center*

Domain Constraints:

- The attributes *start_shift_time* and *end_shift_time* must have datetime values
- The value of the attribute *start_shift_time* must be an earlier datetime than the value of *end_shift_time*

Relationship: *has*
Structural Constraints (Cardinality and Participation Constraints):

- Relationship of *Voting_Center-has-Operating_Period* is one-to-many
- Total participation from *Voting_Center* and partial participation from *Operating_Period*

Other Constraints:

- An operating period can be modified only if nobody has registered to vote at the associated center during the time period

Relationship: *connected_to*
Structural Constraints (Cardinality and Participation Constraints):

- Relationship of *Voting_Registration-connected_to-Voting_Center* is many-to-one
- Total participation from *Voting_Registration* and partial participation from *Voting_Center*

Relationship: *registers_for*
Structural Constraints (Cardinality and Participation Constraints):

- Relationship of *Folks-registers_for-Voting_Registration* is one-to-many
- Total participation from *Voting_Registration* and partial participation from *Folks*

Relationship: *enabled_by*
Structural Constraints (Cardinality and Participation Constraints):

- Relationship of *Ballot-enabled_by-Voting_Registration* is many-to-one
- Total participation from *Ballot* and partial participation from *Voting_Registration*

Relationship: *cast_to*
Structural Constraints (Cardinality and Participation Constraints):

- Relationship of *Ballot-cast_to-Poll* is many-to-one

- Total participation from *Ballot* and partial participation from *Poll*

Relationship: *created_by*
Structural Constraints (Cardinality and Participation Constraints):
- Relationship of *Ballot-created_by-Folks* is one-to-many
- Total participation from *Ballot* and partial participation from *Folks*

Relationship: *cast_at*
Structural Constraints (Cardinality and Participation Constraints):
- Relationship of *Ballot-cast_at-Voting_Center* is many-to-one
- Total participation from *Ballot* and partial participation from *Voting_Center*

Reasons for Design

Folk Entity:

To start my Entity-Relationship diagram, I created the entity *Folks*, which has the attributes *folk_id, name (first_name, last_name, nickname), primary_phone_number, secondary_phone_number, and email*. I created these attributes based on the data requirements stated in the project description. The attribute *folk_id* is used as the primary key for this entity, because it uniquely identifies each entity instance. Due to this, *folk_id* has unique and not-null constraints, and must be exactly 16 digits, which adheres to the project data requirements. Also, I declared the attribute *name* as a composite attribute, because a folk's name is made up of their *first_name, last_name, and nickname* according to the project data requirements. Moreover, since a folk can have one or more email addresses, I declared *email* as a multi-value attribute. Additionally, the value of *email* must contain the special character, @, because that is how emails are formed in real-life. Some other constraints for this entity is that the values for the attributes *primary_phone_number* and *email* must be unique and not-null, as folks should have their own phone number and email address. This is because, in a real-world scenario, the Elections Department needs to have multiple ways of contacting a specific folk. Moreover, *first_name* and *last_name* cannot be null because every folk should have a full name. Also, the values for *first_name, last_name, and nickname* should be all letters, with no numbers or special characters except for dash, because that is how names are in real-life. Furthermore, the value of the attributes *primary_phone_number* and *secondary_phone_number* must be exactly 10 digit numbers, which adheres to the description in the project data requirements. Also, I did not put a not-null constraint on *secondary_phone_number* because, realistically, not all folks have two different phones. Lastly, the *Folks* entity can be specialized via ISA to the entity *Elections_Staff*. This is because, according to the project requirements, some folks are staff members of the Elections Department.

Elections_Staff Entity:

The *Elections_Staff* entity is a specialization, via the ISA relationship, of the entity *Folks*. Thus, *Elections_Staff* will inherit all of the attributes of *Folks*. As a result, the primary key of *Elections_Staff* is *folk_id*, which uniquely identifies the staff member. *Elections_staff* has the attribute *staff_role* which cannot be null because every staff member has a role. This attribute is used to identify the role of the staff, which has the domain

constraint of being either “clerk” or “monitor”, because those are the staff member roles according to the project data requirements.

Place Entity:

Next, I created the entity *Place*, which has the attributes *place_id, address (street [street_number, street_name], city, state, zip_code), x_coordinate, y_coordinate*. I created these attributes based on the data requirements stated in the project description. The attribute *place_id* is used as the primary key for this entity, and will be used to uniquely identify each entity instance. Due to this, *place_id* has unique and not-null constraints. The value of *place_id* must be alphanumeric and 10 characters, which allows for uniqueness and easy identification. Also, I declared the attribute *address* as a composite attribute, because a place’s address is made up of their *street_number, street_name, city, state,* and *zip_code* according to the project data requirements. Additionally, to make it similar to how addresses are made in the real-world, I declared the domain constraints that *street_number* and *zip_code* must be an integer, and *street_name, city,* and *state* must be alphabetical characters with the exception of dashes. Moreover, another domain constraint is that the value of the attributes *x_coordinate* and *y_coordinate* must be floats and fall within the valid ranges. This is because coordinates are floats according to the project description. Furthermore, none of the attributes in this entity are allowed to be null because, in the project data requirements, it says, “Each place in Wonderland has an address (street number, street name, city, state, and zipcode) and XY-coordinates (a pair of floats; see below)”. Thus, all attributes need to have a value, which is also in conjunction with real-life, as all addresses need these values to be properly identified. Additionally, according to the requirements, no two distinct places in Wonderland can have the same address or coordinates. As a result, I made the constraint that the attributes *address,* and *x_coordinate* and *y_coordinate* pair has to be unique for each entity instance. Lastly, according to the project requirements, a place is either a residence or voting center, but not both. Therefore, I created a total disjoint ISA relationship where *Place* is specialized to be either a *Voting_Center* or *Residence* entity.

Voting_Center Entity:

The *Voting_Center* entity is a specialization, via the ISA relationship, of the entity *Place*. Thus, *Voting_Center* will inherit all of the attributes of *Place*. As a result, the primary key of *Voting_Center* is *place_id*, which uniquely identifies the place in Wonderland. *Voting_Center* has the attribute *acronym* which is unique, cannot be null, and has exactly 4 alphanumeric characters. This is in adherence to the project requirements, which states each voting center in Wonderland has a unique acronym made up of 4 alphanumeric characters. All voting centers need this attribute as it will be used to identify them. Lastly, another constraint on *Voting_Center* involves the entity’s relationship to *Operating_Period*. According to the requirements, a voting center’s operating period can only be modified only if nobody has registered to vote at the center during the associated time period. Therefore, I stated that as a constraint to adhere to the requirements. This also makes sense for a real-world situation, as voting centers cannot change their hours if people already registered to vote there at that time. It would be a major hassle and major disservice to notify people to register again for a different time.

Residence Entity:

The *Residence* entity is a specialization, via the ISA relationship, of the entity *Place*. Thus, *Residence* will inherit all of the attributes of *Place*. As a result, the primary key of *Residence* is *place_id*, which uniquely identifies the place in Wonderland. *Residence* has the attribute *household_size* which counts the amount of folks that live in that place. I created this entity because each folk lives in a residence in Wonderland. Also, I created

the attribute *household_size* because many folks can live in the same residence, and it is important for the Elections Department to know how many people live in a residence. Thus, the *household_size* cannot be null, and the value must be an integer because it is a count.

Operating_Period **Entity:**

I created the entity *Operating_Period* with the attributes *op_id, start_datetime, and end_datetime*. I decided to make the operating periods of the voting centers a separate entity, instead of just attributes of *Voting_Center*, because each center can have one or more operating periods according to the project requirements. This way, through the relationship set *has*, a voting center can reference its multiple unique operating periods with the primary key, *op_id*. Also, the primary key *op_id* is used to uniquely identify a voting center's specific operating periods. By using the primary key as a unique identifier, multiple voting centers may be able to have the same *start_datetime* and *end_datetime* as other centers, but each center will reference its own instance in *Operating_Period*. This allows each *Voting_Center* to have their own specific periods tied to them. This protects against affecting other center's operating periods, if a voting center has to modify their own operating periods. Moreover, to adhere to the project requirements, there is a domain constraint on the attributes *start_datetime* and *end_datetime*, where they must have datetime values. Furthermore, to adhere to logical constraints, the value of the attribute *start_datetime* must be an earlier date than the value of *end_datetime*. Additionally, the attributes *start_datetime* and *end_datetime* must not be null because each operating period needs both to be a proper interval. Lastly, another constraint is that an operating period can be modified only if nobody has registered to vote at the associated center during the time period. This is in accordance with the project requirements.

Poll **Entity:**

I created the entity *Poll*, which has the attributes *poll_name_id, question_text, start_datetime_availability* and *end_datetime_availability*. I created these attributes based on the data requirements stated in the project description. The attribute *poll_name_id* is used as the primary key for this entity, and will be used to uniquely identify each entity instance. Due to this, *poll_name_id* has unique and not-null constraints. To further adhere to the project data requirements, I added the domain constraints that the value of the attribute *poll_name_id* must be exactly 4 alphanumeric characters and the attributes *start_datetime_availability* and *end_datetime_availability* must have datetime values. Furthermore, to adhere to logic, the value of the attribute *start_datetime_availability* must be an earlier date than the value of *end_datetime_availability*. Furthermore, none of the attributes in this entity are allowed to be null because, in the project data requirements, it says, "Each poll has a unique short name (with 4 alphanumeric characters [A-Z0-9]), a question text, and an availability period (i.e., the datetime interval when folks may cast a ballot vote for the poll). Thus, all attributes need to have a value, which is also in conjunction with real-life, as all polls need identifiers, questions, and availability periods. Lastly, I added the modification constraint that a poll cannot be modified if ballots exist for it and the deletion constraint that a poll can only be deleted if it has no registrations. These constraints are needed to adhere to the project requirements. In the real-world, polls cannot be modified if people already submitted ballots for them, and polls cannot be deleted if people already registered for them.

Ballot **Entity:**

I created the entity *Ballot*, which has the attributes *ballot_id, voter's_choice, cast_datetime* and *validation_status*. I created these attributes based on the data requirements stated in the project description, "For each casted ballot, the Elections department wishes to record the voter's choice (vote), the datetime when the ballot was cast, and information uniquely identifying the corresponding valid voting registration (that enabled the vote)". The attribute *ballot_id* is used as the primary key for this entity, and will be used to uniquely identify each entity instance. Due to this, *ballot_id* has unique and not-null constraints. To further adhere to the project data requirements, I added the domain constraints that the *cast_datetime* must be a datetime value and the value of the *voter's_choice* attribute must be "YES", "NO", "ABSTAIN", or "NULL". Also, I added the domain constraint that the value of the *validation_status* attribute must be "VALID" or "INVALID". This will be used to identify if the ballot is associated with a valid registration and is to be counted in a poll. Thus, this attribute cannot be null. Another constraint is that the attribute *cast_datetime* must be within the associated Voting_Center's Operating_Period. It would not make sense for the ballot to be cast outside of the voting center's operating hours.

Voting_Registration Entity:

I created the entity *Voting_Registration*, which has the attributes *registration_id, desired_voting_date,* and *validation_status*. I created these attributes based on the data requirements stated in the project description. In order for a folk to register, they must provide a desired voting date, hence the not-null constraint, and a voting center they want to vote at. The desired voting center of the folk is not an explicit attribute because it is displayed through the relationship *Voting_Registration-connected_to-Voting_Center*. The attribute *validation_status* is used to state whether the folk's registration is valid after checking if the *desired_voting_date* is within the operating periods of the voting center. Only then is the registration valid. As a result, *validation_status* has a not-null constraint because it is needed to determine if a folk is properly registered to vote. This is in conjunction to real-world scenarios, where a person needs to be properly registered to vote in elections. Moreover, *validation_status* has a domain constraint that the value of the attribute must be "VALID" or "INVALID". Furthermore, the attribute *registration_id* is used as the primary key for this entity, and will be used to uniquely identify each entity instance. Due to this, *registration_id* has unique and not-null constraints. This attribute will be used to uniquely identify each registration that a folk makes, since a folk can have multiple valid registrations according to the project requirements.

lives_in Relationship:

I created the relationship set *lives_in*, which involves the entities *Folks* and *Residence*. This relationship represents the description given in the project requirements, "All folks of Wonderland reside at a place, and some folks may reside at the same place." This relationship shows total participation from *Folks* because all folks live in a place. However, the project requirements do not say that every *Residence* has folks living in it. Thus, this relationship shows partial participation from *Residence*. Lastly, the relationship *Folks-lives_in-Residence* is many-to-one because a folk is associated with one residence, and one residence can hold multiple folks. This design is coherent with the real-world.

works_at Relationship:

I created the relationship set *works_at*, which involves the entities *Elections_Staff* and *Voting_Center*. This relationship set represents the description given in the project requirements, "Elections staff are scheduled to work at a voting center for some datetime interval. Some staff may not be scheduled at all." This relationship shows partial participation from *Elections_Staff* because, according to the description, some staff may not have a schedule to work at a voting center. There is total participation from *Voting_Center* because all voting centers need staff members, or else they will not function. Moreover, this relationship set has the attributes *start_shift_time* and *end_shift_time*. These attributes are needed because they represent the staff members' work schedules. The staff members that do have a schedule need a designated start and end datetime to represent their shift. Additionally, in accordance with the project requirements, the attributes *start_shift_time* and *end_shift_time* must have datetime values as a domain constraint. Also, as a logical domain constraint, the value of the attribute *start_shift_time* must be an earlier datetime than the value of *end_shift_time*. Lastly, the relationship *Elections_Staff-works_at-Voting_Center* is many-to-many because I designed the relationship so that an elections staff member may work at many voting centers, and a voting center has many staff members. In real-world scenarios, staff members can realistically be assigned to different voting centers on different days. Therefore, I did not make the relationship many-to-one, where each staff member is assigned to one center.

has **Relationship:**

I created the relationship set *has*, which involves the entities *Voting_Center* and *Operation_Period*. This relationship set represents the description given in the project requirements, "Voting centers have…one or more operating periods, i.e., a collection of datetime intervals." This relationship shows total participation from *Voting_Center* because each center is required to have at least one operating period. There is only partial participation from *Operating_Period* because some unique operating periods that were created as options may not be used by a center. Lastly, the relationship *Voting_Center-has-Operating_Period* is one-to-many because a voting center can have many operating intervals, but an *Operating_Period* instance is uniquely associated with one voting center. This is so that voting centers that happen to have the same operating period can freely modify their own when needed without interfering with the operating periods of other centers. Also, to adhere to the project requirements, an operating period can be modified only if nobody has registered to vote at the associated center during the time period.

connected_to **Relationship:**

I created the relationship set *connected_to*, which involves the entities *Voting_Registration* and *Voting_Center*. This relationship set represents the description given in the project requirements, "Wonderland forks indicate their intent to cast a ballot for a poll at a voting center on some date by registering with the Elections department. To register, they provide their identifier, the voting center identifier, and their desired voting date." This relationship shows total participation from *Voting_Registration* because each registration has to be tied to a specific *Voting_Center* according to the description. There is only partial participation from *Voting_Center* because a center might exist that does not have any registrations associated with it. Lastly, the relationship *Voting_Registration-connected_to-Voting_Center* is many-to-one because each registration is linked to one specific voting center, and one voting center can have many registrations tied to it. This design is coherent with the real-world.

registers_for **Relationship:**

I created the relationship set *registers_for*, which involves the entities *Folks* and *Voting_Registration*. This relationship set represents the description given in the project requirements, "Wonderland forks indicate their intent to cast a ballot for a poll at a voting center on some date by registering with the Elections department…A folk can have multiple valid registrations." This relationship shows partial participation from *Folks* because not every folk is required to vote, therefore, not every folk will register to vote. Some folks might not want to vote. However, there is total participation from *Voting_Registration* because each registration is completed by a folk. Lastly, the relationship *Folks-registers_for-Voting_Registration* is one-to-many because, according to the description, a folk can have many valid registrations, and each registration is uniquely associated to one folk.

enabled_by **Relationship:**

I created the relationship set *enabled_by*, which involves the entities *Ballot* and *Voting_Registration*. This relationship set represents the description given in the project requirements, "A folk can cast a ballot (vote) for a poll on the date and center of one of their valid registrations, and during the operating hours of that center on that date…For each casted ballot, the Elections department wishes to record the voter's choice (vote), the datetime when the ballot was cast, and information uniquely identifying the corresponding valid voting registration (that enabled the vote)." This relationship set shows that ballots need to be enabled by a valid voter registration before they are cast to polls. This relationship shows total participation from *Ballot* because each ballot has to be enabled by a valid registration. Also, this relationship shows partial participation from *Voting_Registration* because not every registration produces a ballot. Some folks may register to vote, but not actually vote. Lastly, the relationship *Ballot-enabled_by-Voting_Registration* is many-to-one because there are many different polls, hence many different ballots that need to be validated with one voting registration from a folk. Also, one registration yields ballots for multiple polls, thus connecting to many different ballots.

cast_to **Relationship:**

I created the relationship set *cast_to*, which involves the entities *Ballot* and *Poll*. This relationship set represents the description given in the project requirements, "A folk can cast a ballot (vote) for a poll" This relationship set shows that ballots are cast to polls. There is total participation from *Ballot* in this relationship because each ballot is cast to one specific poll. However, there is only partial participation from *Poll* because a poll can exist without ballots being cast into it. A new poll could just be created, which will not have any ballots cast to it. Lastly, the relationship *Ballot-cast_to-Poll* is many-to-one because each ballot corresponds to one specific poll, and one poll can have many ballots cast to it. This design is coherent to real-world scenarios.

created_by **Relationship:**

I created the relationship set *created_by*, which involves the entities *Ballot* and *Folks*. This relationship set represents the description given in the project requirements, "A folk can cast a ballot (vote) for a poll on the date and center of one of their valid registrations, and during the operating hours of that center on that date." This relationship set is needed to show that ballots exist because folks create them. There is total participation from *Ballot* in this relationship because each ballot is created by a folk. Logically, a ballot possibly cannot exist without being produced by a folk. However, there is only partial participation from *Folks* in this relationship because not every folk has to create a ballot. Realistically, some folks might not want to participate in the elections, and thus, not every folk has to participate in creating a ballot. Lastly, the relationship *Ballot-created_by-Folks* is one-to-one because each individual ballot is created by one folk. Each ballot is associated with one folk, thus the relationship is one-to-one.

cast_at Relationship:

I create this relationship set *cast_at*, which involves the entities *Ballot* and *Voting_Center*. This relationship set represents the description given in the project requirements, "When a folk casts a ballot for the poll, the Elections Department maintains their answer to the question of the poll together with the datetime and center where they casted the vote." This relationship set shows that each ballot is associated with one specific voting center. Every ballot is required to contain information regarding the center where it was cast. This is because each ballot is associated with a voting registration, which is connected to a specific voting center. Each ballot can only be cast at the voting center that is chosen in the registration. There is total participation from *Ballot* because every ballot needs a voting center, as it is cast there. However, there is only partial participation from *Voting_Center* in this relationship because a voting center can exist without ballots being cast to it. Lastly, the relationship set *Ballot-cast_at-Voting_Center* is many-to-one because each ballot is tied to one voting center, and one voting center can have many ballots cast to it.

Phase C Report:

C. Logical Design

This stage involves the mapping of your conceptual design above onto the relational data model. In this stage, you will design the tables for all your entities and relations. You should apply all normalizations you find useful and/or necessary. Ensure that your design still satisfies the user requirements. Justify your design choices.

List of Relations

Folks(folk_id, first_name, last_name, nickname, date_of_birth, primary_phone_number, secondary_phone_number, place_id)
Primary Key: folk_id
Foreign Key Reference Constraint: Folks(place_id) → Residence(place_id) ON DELETE RESTRICT

Folks_Email(folk_id, email)
Primary Key: folk_id, email
Foreign Key Reference Constraint: Folks_Email(folk_id) → Folks(folk_id) ON DELETE CASCADE

Elections_Staff(folk_id, staff_role)
Primary Key: folk_id
Foreign Key Reference Constraint: Elections_Staff(folk_id) → Folks(folk_id) ON DELETE CASCADE

Place(place_id, street_number, street_name, city, state, zip_code, x_coordinate, y_coordinate)
Primary Key: place_id

Voting_Center(place_id, acronym)
Primary Key: place_id
Foreign Key Reference Constraint: Voting_Center(place_id) → Place(place_id) ON DELETE CASCADE

Residence(place_id, household_size)
Primary Key: place_id
Foreign Key Reference Constraint: Residence(place_id) → Place(place_id) ON DELETE CASCADE

Operating_Period(op_id, start_datetime, end_datetime, place_id)
Primary Key: op_id
Foreign Key Reference Constraint: Operating_Period(place_id) → Voting_Center(place_id) ON DELETE CASCADE

Works_At(folk_id, place_id, start_shift_datetime, end_shift_datetime)
Primary Key: folk_id, place_id, start_shift_datetime
Foreign Key Reference Constraint: Works_At(folk_id) → Elections_Staff(folk_id) ON DELETE CASCADE
Foreign Key Reference Constraint: Works_At(place_id) → Voting_Center(place_id) ON DELETE CASCADE

Voting_Registration(registration_id, desired_voting_date, validation_status, folk_id, center_id)
Primary Key: registration_id
Foreign Key Reference Constraint: Voting_Registration(folk_id) → Folks(folk_id) ON DELETE CASCADE
Foreign Key Reference Constraint: Voting_Registration(center_id) → Voting_Center(place_id) ON DELETE CASCADE

Ballot(ballot_id, voter's_choice, cast_datetime, ballot_status, registration_id, poll_name_id, folk_id, center_id)
Primary Key: ballot_id
Foreign Key Reference Constraint: Ballot(registration_id) → Voting_Registration(registration_id) ON DELETE RESTRICT
Foreign Key Reference Constraint: Ballot(poll_name_id) → Poll(poll_name_id) ON DELETE RESTRICT
Foreign Key Reference Constraint: Ballot(folk_id) → Folks(folk_id) ON DELETE RESTRICT
Foreign Key Reference Constraint: Ballot(center_id) → Voting_Center(place_id) ON DELETE RESTRICT

Poll(poll_name_id, question_text, start_datetime_availability, end_datetime_availability)
Primary Key: poll_name_id

Justifications

Folks and Folks_Email Relation:
In order to map the *Folks* entity from my ER diagram to the relational model, I created the *Folks* relation with the attributes *folk_id, first_name, last_name, nickname, primary_phone_number, secondary_phone_number, and place_id.* The primary key of the relation will be *folk_id* which is the same as in the ER diagram. This attribute will be used as a unique identifier for each tuple in the *Folks* relation. The composite attribute *name* in the ER diagram is broken down to its simple, individual components, *first_name, last_name, and nickname* in the relational model. The attributes *primary_phone_number* and *secondary_phone_number* are already simple attributes in the ER model so they are directly added to the relational model without any change. However, the attribute *email* in the ER diagram is a multivalued attribute because a *Folk* can have multiple emails. To implement this in the relational model, I had to create a separate relation *Folks_Email*. The primary key of *Folks_Email* will be composed of the attributes *folk_id* and *email* with a foreign key reference constraint *Folks_Email*(*folk_id*) → *Folks*(*folk_id*). This way, one *Folk* is able to have multiple emails and each email can be directly associated with their *Folk*. Lastly, the attribute *place_id* is a foreign key to the attribute *place_id* in the *Residence* relation. This foreign key constraint in the relational model for the *Folks* relation is used to show the many-to-one *lives_in* relationship in the ER diagram. With this foreign key constraint, each tuple in the *Folks* relation is associated with a tuple in the *Residence* relation, which correctly shows that each *Folk* in Wonderland lives in a residence. In SQL, I will create a not null constraint for this attribute to enforce total participation from each tuple in the *Folks* relation. Also, the ON DELETE action is RESTRICT because a residence should not be deleted if folks live there.

Elections_Staff Relation:
In order to map the ISA specialization between the *Folks* entity and *Elections_Staff* entity in the ER model, I created a relation named *Elections_Staff*. The primary key of this relation will be *folk_id* with a foreign key reference constraint *Folks_Email*(*folk_id*) → *Folks*(*folk_id*) ON DELETE CASCADE. This relational design correctly implements the inheritance between the *Elections_Staff* entity and *Folks* entity, where every member of the *Elections_Staff* relation is also a member of the *Folks* relation by adding the foreign key reference constraint and having *folk_id* as the primary key as well. Lastly, this design satisfies normalization because it creates a separate relation for *Elections_Staff* with its own attribute of *staff_role*. This attribute is only relevant to staff members, thus a separate relation is needed.

Place Relation:
In order to map the *Place* entity from my ER diagram to the relational model, I created the *Place* relation with the attributes *place_id, street_number, street_name, city, state, zip_code, x_coordinate,* and *y_coordinate.* The primary key of the relation will be *place_id* which is the same as in the ER diagram. This attribute will be used as a unique identifier for each tuple in the *Place* relation. The composite attribute *address* and *street* in the ER diagram is broken down to its simple, individual components, *street_number, street_name, city, state,* and *zip_code*. The attributes *x_coordinate* and *y_coordinate* are already simple attributes in the ER model so they are directly added to the relational model without any change. In order to show the ISA total disjoint specialization relationship between *Place* → *Voting_Center* or *Residence*, I created separate relations for *Voting_Center* and *Residence*, where both relations will use a foreign key reference constraint to the primary key of *Place*, which is *place_id*. To implement total disjointness, I will create a trigger or check constraint in SQL for this specialization.

Voting_Center Relation:
In order to map the ISA specialization between the *Place* entity and *Voting_Center* entity in the ER model, I created a relation named *Voting_Center*. The primary key of this relation will be *place_id* with a foreign key reference constraint *Voting_Center*(*place_id*) → *Place*(*place_id*) ON DELETE CASCADE. This relational design correctly implements the inheritance between the *Voting_Center* entity and *Place* entity, where every member of the *Voting_Center* relation is also a member of the *Place* relation by adding the foreign key reference constraint and having *place_id* as the primary key as well. Through this design, a *Voting_Center* inherits all of the attributes of the *Place* relation as it is a *Place* itself. Lastly, this design satisfies normalization because it creates a separate relation for *Voting_Center* with its own attribute of *acronym*. This attribute is only relevant to voting centers, thus a separate relation is needed.

Residence Relation:
In order to map the ISA specialization between the *Place* entity and *Residence* entity in the ER model, I created a relation named *Residence*. The primary key of this relation will be *place_id* with a foreign key reference constraint *Residence*(*place_id*) → *Place*(*place_id*) ON DELETE CASCADE. This relational design correctly implements the inheritance between the *Residence* entity and *Place* entity, where every member of the *Residence* relation is also a member of the *Place* relation by adding the foreign key reference constraint and having *place_id* as the primary key as well. Through this design, a *Residence* inherits all of the attributes of the *Place* relation as it is a *Place* itself. Lastly, this design satisfies normalization because it creates a separate relation for *Residence* with its own attribute of *household_size*. This attribute is only relevant to residences, thus a separate relation is needed.

Operating_Period Relation:
In order to map the *Operating_Period* entity from my ER diagram to the relational model, I created the *Operating_Period* relation with the attributes *op_id, start_datetime, end_datetime,* and *place_id*. The primary key of the relational will be *op_id*, which is the same as in the ER diagram. This attribute will be used as a unique identifier for each tuple in the *Operating_Period* relation. The attributes *start_datetime* and *end_datetime* are already simple attributes in the ER model so they are directly added to the relational model without any change. Additionally, the last attribute in this relation is *place_id* which is a foreign key to the attribute *place_id* in the *Voting_Center* relation. There will be a foreign key reference constraint *Operating_Period(place_id) → Voting_Center(place_id)* ON DELETE CASCADE. This foreign key constraint in the relational model for the *Operating_Period* relation is used to show the one-to-many *Voting_Center-has-Operating_Period* relationship in the ER diagram. With this foreign key reference constraint, each tuple in the *Operating_Period* relation is associated with one tuple in the *Voting_Center* relation, which correctly shows that each unique operating period is linked to one voting center. Also, this foreign key reference constraint correctly allows each voting center to have one or more operating periods. I decided on this design so that each voting center has unique operating periods. Two tuples in the *Operating_Period* relation can have the same *start_datetime* and *end_datetime*, but they can be uniquely identified with their *op_id*. This way, each voting center has operating periods that can be potentially edited or deleted if need be, without incorrectly editing or deleting the operating period of another voting center.

Works_At Relation:
When mapping my ER diagram to a relational schema, I needed to create a relation to represent the relationship *works_at* in the ER diagram. This is because the relationship *Elections_Staff-works_at-Voting_Center* in the ER diagram is a many-to-many relationship. To represent a many-to-many relationship in the relational model, a separate relation is needed. The primary key for this relation will be composed of two attributes, *folk_id* and *place_id*, with the foreign key reference constraints *Works_At(folk_id) → Elections_Staff(folk_id)* and *Works_At(place_id) → Voting_Center(place_id)*. Through this design, *Works_At* acts as a junction table, where each tuple in the relation represents a single, specific association between one election staff member and one voting center. This correctly shows the many-to-many relationship, that one staff member can work at multiple different voting centers, and one voting center has many staff members working there. Furthermore, the foreign key reference constraints ensure referential integrity, where every tuple in the *Works_At* relation corresponds to a valid staff member and valid voting center. Lastly, this relation has the attributes *start_shift_time* and *end_shift_time*, which correspond to the attributes for the relationship set *works_at*. These attributes will be used to show the shift interval of the *Elections_Staff* tuple at a specific *Voting_Center*.

Voting_Registration Relation:
In order to map the *Voting_Registration* entity from my ER diagram to the relational model, I created the *Voting_Registration* relation with the attributes *registration_id, desired_voting_date, validation_status, folk_id,* and *center_id*. The primary key of the relational will be *registration_id*, which is the same as in the ER diagram. This attribute will be used as a unique identifier for each tuple in the *Voting_Registration* relation. The attributes *desired_voting_date* and *validation_status* are already simple attributes in the ER model so they are directly added to the relational model without any change. The attribute *desired_voting_date* will store a datetime value that indicates the date that the folk wants to place their vote on. The attribute *validation_status* will either have the value “valid” or “invalid”. A voting registration will only be valid if the *desired_voting_date* is contained within the specified voting center. Moreover, the attribute *folk_id* is a foreign key to attribute *folk_id* in the

relation *Folks*. There will be a foreign key reference constraint *Voting_Registration(folk_id) → Folks(folk_id)* ON DELETE CASCADE. This foreign key constraint for the *Voting_Registration* relation is used to show the one-to-many relationship in the ER diagram, *Folks-registers_for-Voting_Registration*. This foreign key constraint shows that one folk can have multiple voting registrations, and that one voting registration is associated with one folk. Additionally, the attribute *center_id* is a foreign key to the attribute *place_id* in the relation *Voting_Center*. There will be a foreign key reference constraint *Voting_Registration(place_id) → Voting_Center(place_id)* ON DELETE CASCADE. This foreign key constraint for the *Voting_Registration* relation is used to show the many-to-one relationship in the ER diagram, *Voting_Registration-connected_to-Voting_Center*. This foreign key constraint shows that many registrations can be associated with one voting center, and that one unique registration is associated with only one voting center. This foreign key constraint is needed to satisfy the requirements in the project description. Also, both of these foreign keys cannot be null, as a voting registration needs to have a folk, because a folk creates the registration, and a linked voting center, because every registration has to be associated with one.

Ballot Relation:

In order to map the *Ballot* entity from the ER diagram to the relational model, I created the *Ballot* relation with the attributes *ballot_id, voter's_choice, cast_datetime, validation_status, registration_id,* and *poll_name_id*. The primary key of the relational will be *ballot_id*, which is the same as in the ER diagram. This attribute will be used as a unique identifier for each tuple in the *Ballot* relation. The attributes *voter's_choice*, *cast_datetime,* and *validation_status* are already simple attributes in the ER model so they are directly added to the relational model without any change. The attribute *voter's_choice* will store the value of the answer that a Folk made for a poll. The possible answers are "YES", "NO", "ABSTAIN", or "NULL". The attribute *cast_datetime* will have a datetime value which will indicate the time the ballot was cast. The attribute *validation_status* will either have the value "valid" or "invalid". A ballot will only be valid if it is linked to a valid tuple in the *Voting_Registration* relation. Moreover, the attribute *registration_id* is a foreign key to attribute *registration_id* in the relation *Voting_Registration*. There will be a foreign key reference constraint *Ballot(registration_id) → Voting_Registration(registration_id)* ON DELETE CASCADE. This foreign key constraint for the *Ballot* relation is used to show the many-to-one relationship in the ER diagram, *Ballot-enabled_by-Voting_Registration*. This foreign key constraint shows that many different ballots can be validated with one voting registration from a folk because there are many different polls. Also, it shows that one ballot is associated with one registration. Furthermore, the attribute *poll_name_id* is a foreign key to attribute *poll_name_id* in the relation *Poll*. There will be a foreign key reference constraint *Ballot(poll_name_id) → Poll(poll_name_id)* ON DELETE CASCADE. This foreign key constraint for the *Ballot* relation is used to show the many-to-one relationship in the ER diagram, *Ballot-cast_to-Poll*. This foreign key constraint shows that one poll can have many ballots cast to it, and that a ballot is cast to one specific poll. Also, both of these foreign keys cannot be null, as a ballot needs to be validated by a voting registration, and a ballot needs to specify the poll it is being cast to.

Poll Relation:

In order to map the *Poll* entity from the ER diagram to the relational model, I created the *Poll* relation with the attributes *poll_name_id, question_text, start_datetime_availability,* and *end_datetime_availability*. The primary key of the relational will be *poll_name_id*, which is the same as in the ER diagram. This attribute will be used as a unique identifier for each tuple in the *Poll* relation. The attributes *question_text*, *start_datetime_availability,* and *end_datetime_availability* are already simple attributes in the ER model so they are directly added to the relational model without any change. The attribute *question_text* will store the question

of the poll. The attributes *start_datetime_availability* and *end_datetime_availability* will store the availability interval for the poll.

Phase D Report:

D. Physical Design This stage involves the following:
• Design a MySQL database based on the design developed in section B.
• Implement tables for the relations and the constraints. Maintain scripts for the creation and deletion of tables.
• Maintain scripts for loading data into your tables.
• Ensure that your design still satisfies the user requirements.
• Justify your design choices.

Justification: createAll.sql

I created all of the relations using the SQL command "CREATE TABLE IF NOT EXISTS [relation_name] ([attributes]);" This command is used to create relations in SQL. Using this command, I defined each relations' attributes, data types, keys, and constraints.

Place Relation:
The first relation that I created was the *Place* table. I created this relation before all the rest because it does not have any foreign key constraints and it is a parent table to the *Voting_Center* and *Residence* relations. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Place* schema from Phase C. I decided to make the primary key's data type "INT AUTO_INCREMENT" because integers are all unique, making them perfect to differentiate between tuples. Also, AUTO_INCREMENT is an easy way for each place in Wonderland to have a primary key without having to explicitly define each one. It makes INSERT statements easier, where the user does not have to find an integer that has not been used. The database will create a primary key for them. For the *street_number, street_name, city, and state* attributes I used VARCHAR() as the data type because each of these attributes will hold non-fixed-length strings as their values. However, *zip_code* is CHAR(5) because all zip codes are strings of a fixed-length 5. I decided to make the data type of *x_coordinate* and *y_coordinate* to be DECIMAL(10, 7) to ensure precision. Using floats can introduce rounding errors that can affect the distance calculation between two tuples in the *Place* entity. The constraints "*UNIQUE (street_number, street_name, city, state, zip_code)*" and "*UNIQUE (x_coordinate, y_coordinate)*" are used to satisfy the project requirements that no two distinct places can have the same address and coordinates. These constraints group together the attributes that compose the address and coordinates of a place, and enforces a unique constraint of the groups. Lastly, each attribute has a not null constraint because all tuples of the *Place* relation must have all of these attributes according to the project description.

Voting_Center Relation:
I then created the *Voting_Center* relation which is a subclass of the *Place* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Voting_Center* schema from Phase C. The primary key of this relation is *place_id* with a foreign key reference to *Place*(*place_id*), which correctly implements the ISA specialization relationship between *Place* and *Voting_Center*. The foreign key constraint is set to ON DELETE CASCADE, which deletes the place from the *Voting_Center* relation if it is also deleted from the *Place* relation. This ensures that a place does not exist in

Voting_Center if it does not exist in *Place*. Also, The attribute *acronym* has the data type CHAR(4) because of the description in the project requirements, "Voting centers have unique acronyms (with 4 alphanumeric characters [A-Z0-9])". Moreover, to make sure that the values of the attribute only have alphanumeric characters [A-Z0-9], I implemented the constraint which checks that the inserted acronym only consists of 4 alphanumeric characters. Lastly, I enforce a NOT NULL and UNIQUE constraint on acronym because each voting center has a unique acronym according to the project requirements.

Residence Relation:
I then created the *Residence* relation which is a subclass of the *Place* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Residence* schema from Phase C. The primary key of this relation is *place_id* with a foreign key reference to *Place*(*place_id*), which correctly implements the ISA specialization relationship between *Place* and *Residence*. The foreign key constraint is set to ON DELETE CASCADE, which deletes the place from the *Residence* relation if it is also deleted from the *Place* relation. This ensures that a place does not exist in *Residence* if it does not exist in *Place*. Lastly, the attribute *household_size* is an integer because it is just an integer that states how many folks live in a residence. There are no decimals, thus it is an integer.

Folks Relation:
I then created the *Folks* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Folks* schema from Phase C.
I created this relation before *Folks_Email* and *Elections_Staff* because both of these relations have a foreign key constraint to *Folks*. I decided to make the data type of the primary key *folk_id* to be CHAR(16) to adhere to the requirement in the project description, "The information stored about each individual in Wonderland includes a 16-digit personal identification number". CHAR(16) makes each *folk_id* a fixed-length of 16. Moreover, the attributes *first_name, last_name,* and *nickname* are all VARCHAR(50) because their values are not a fixed length. I decided to enforce a NOT NULL constraint on *first_name* and *last_name* because every folk needs to have one of each to create a full name. However, I did not enforce the same constraint on *nickname*, because some folks might not have *nickname*. The date_of_birth is of data type DATE with a not null constraint because it is impossible for a folk to not have a date of birth. Moreover, the attributes *primary_phone_number* and *secondary_phone_number* are CHAR(10) to adhere to the requirement, "The information stored about each individual in Wonderland includes…two 10-digit telephone numbers (primary, and secondary)." CHAR(10) makes the phone numbers of fixed length 10, and these attributes are not declared integers because they will not be used for mathematical calculations. I decided to make *primary_phone_number* have the constraints of not null and unique because each folk needs to have a unique phone number in case the elections department has to contact them directly. I decided that *secondary_phone_number* does not need a not null constraint because some folks might not have another phone number. Furthermore, the attribute *place_id* is a foreign key reference to *Residence(place_id)*. This foreign key is used to represent the relationship *lives_in*, where every folk in Wonderland lives in a residence. This foreign key will be used to associate a folk tuple and the residence they live in. The ON DELETE action is RESTRICT for this foreign key constraint because a residence should not be deleted if folks live there. Moreover, there is a check constraint, "CHECK (folk_id REGEXP '^[0-9]{16}$')" which makes sure that the *folk_id* consists of numbers as stated in the project requirements. Also, there are the check constraints, "CHECK (primary_phone_number REGEXP '^[0-9]{10}$')" and "CHECK (secondary_phone_number REGEXP '^[0-9]{10}$')" which makes sure that the phone number values only contain digits.

Folks_Email Relation:
I then created the *Folks_Email* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Folks_Email* schema from Phase C. This relation is used to represent the project requirement, "The information stored about each individual in Wonderland includes…one or more email addresses." It does this through the attributes *folk_id*, which is a foreign key reference to *Folk(folk_id)*, and *email*, which is where the folk's email will be held. The attribute *email* had a data type of VARCHAR(100) because it is not of fixed length. Also, it has a not null constraint because it would not make sense for an *email* value to be null as that is the purpose of the relation. The primary key is *folk_id* and *email*, which uniquely associates a folk to one of their emails. Through this design, a folk can have one or more emails, and each email is only associated with one folk. Lastly, the foreign key constraint has an action of ON DELETE CASCADE, which deletes the folk and all of their emails from the *Folks_Email* relation if it is also deleted from the *Folk* relation. This ensures that a folk does not exist in *Folks_Email* if it does not exist in *Folks*.

Elections_Staff Relation:
I then created the *Elections_Staff* relation which is a subclass of the *Folks* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Elections_Staff* schema from Phase C. This relation is used to represent the project requirement, "Some folks are staff members of the Elections Department." The primary key of this relation is *folk_id* with a foreign key reference to *Folk*(*folk_id*), which correctly implements the ISA specialization relationship between *Folk* and *Elections_Staff*. The foreign key constraint is set to ON DELETE CASCADE, which deletes the place from the *Elections_Staff* relation if it is also deleted from the *Folk* relation. This ensures that a folk does not exist in *Elections_Staff* if it does not exist in *Folk*. Moreover, the attribute *staff_role* is used to represent the requirement, "Staff can be clerks or monitors." The data type is ENUM('Clerk', 'Monitor') to make sure that these are the only two roles that can be assigned. Lastly, there is a NOT NULL constraint on the *staff_role* attribute because each staff must have one of those two roles.

Operating_Period Relation:
I then created the *Operating_Period* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Operating_Period* schema from Phase C. I created this relation after *Voting_Center* because of the attribute *place_id*, which is a foreign key reference to *Voting_Center(place_id)*. This relation is used to represent the requirement, "Voting centers have…one or more operating periods, i.e., a collection of datetime intervals." Through this design, an operating period can be uniquely associated with one voting center through the foreign key constraint, and a voting center can have one or more periods because it can be referenced as a foreign key by different operating periods. Additionally, this foreign key constraint has the action ON DELETE CASCADE, to ensure that if a voting center is deleted, all of its operating periods are also deleted to ensure data integrity. It would not make sense to have operating periods for a voting center that no longer exists. Also, I decided to make the data type of the primary key INT with AUTO_INCREMENT for simplicity, as all integers are unique and they are great for being used as primary keys. Moreover, AUTO_INCREMENT makes it easier to INSERT new operating periods, as the user does not have to manually check all of the primary keys to see which one is available. Furthermore, *start_datetime* and *end_datetime* have the data type DATETIME to adhere to the description in the project requirements. Also, these attributes have the NOT NULL constraint because it does not make sense for an operating period to not have one of these attributes. If one of these attributes were null, it would not make an interval, and the voting center would not know when to open or close. Lastly, there is a constraint that checks if the *start_datetime* is

before the *end_datetime*. This is needed to adhere to logical constraints, as it does not make sense for a start time to be later than an end time.

Works_At Relation:
I then created the *Works_At* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Works_At* schema from Phase C. I created this relation after *Elections_Staff* and *Voting_Center* because *Works_At* has a foreign key reference constraint to both tables. This relation is used to satisfy the description, "Elections staff are scheduled to work at a voting center for some datetime interval. Some staff may not be scheduled at all." *Works_At* is used to represent the many-to-many relationship between the staff and voting centers. The primary key of this relation is composed of the attributes *folk_id, place_id, and start_shift_datetime*. The attribute *start_shift_datetime* is needed in the primary key, because if not, a staff member would only be allowed to work at a specific voting center once, as a specific *folk_id-place_id* pair would only be allowed as a tuple once. However, by adding *start_shift_datetime* in the primary key, a staff member would be able to work at a voting center for one or many shifts. Furthermore, the attributes *start_shift_datetime* and *end_shift_datetime* have the data type DATETIME to adhere to the project description. Moreover, both of the foreign key reference constraints to *Elections_Staff*(*folk_id*) and *Voting_Center*(*place_id*) have the action ON DELETE CASCADE, because a work shift strictly depends on the existence of both a staff member and voting center. If a staff member or voting center gets deleted, a tuple representing a work shift essentially becomes useless. Thus, the orphaned data must get deleted to ensure data integrity. Lastly, there is a constraint that checks if the *start_shift_datetime* is before the *end_shift_datetime*. This is needed to adhere to logical constraints, as it does not make sense for a start time to be later than an end time.

Poll Relation:
I then created the *Poll* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Poll* schema from Phase C. The primary key *poll_name_id* has a data type of CHAR(4) to adhere to the project description, "Each poll has a unique short name (with 4 alphanumeric characters [A-Z0-9])". CHAR(4) makes sure the poll name has a fixed length of 4 characters, and I also defined a check constraint at the end of the CREATE TABLE declaration to make sure that the poll name is only composed of alphanumeric characters. Moreover, the *question_text* attribute's data type is TEXT because it does not impose a character limit, unlike VARCHAR(). Poll questions can often be lengthy, so a character limit might truncate important words from the question, thus TEXT is a better data type to use over VARCHAR(). Furthermore, the attributes *start_datetime_availability* and *end_datetime_availability* have the data type DATETIME to adhere to the project description. Lastly, both attributes have a NOT NULL constraint because both are needed to create an availability interval.

Voting_Registration Relation:
I then created the *Voting_Registration* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Voting_Registration* schema from Phase C. The primary key of this relation is *registration_id*, which I declared as INT AUTO_INCREMENT. I decided to make the primary key an integer for simplicity, uniqueness, and easy indexing. Also, the AUTO_INCREMENT makes insertions easier for the user. The attribute *desired_voting_date* adheres to the requirement, "To register, they provide…their desired voting date." The data type is DATE because the attribute holds a date value, and it has a NOT NULL constraint because it is needed for a registration. The *validation_status* attribute indicates whether the registration is valid or invalid based on the requirements defined in the project description. The data type is

ENUM('Valid', 'Invalid', 'Pending') to ensure uniformity in indicating whether a registration is valid or invalid. Furthermore, the uniform values in ENUM are utilized by the trigger that checks the validity of the registration. Moreover, this relation is created after *Folks* and *Voting_Center* because *Voting_Registration* contains foreign key references to both of those relations. These constraints are needed to adhere to the requirement, "To register, they provide their identifier, the voting center identifier…" As a result, these attributes also have NOT NULL constraints because they are needed for a registration. The action for these foreign keys are ON DELETE CASCADE, because a tuple in Voting_Registration needs both of these attributes to exist. If one is deleted, the registration should not exist because a folk is needed to create the registration, and a voting center needs to be identified. This maintains data integrity and prevents orphaned data.

Ballot Relation:
I then created the *Ballot* relation. To create this table, I used the "CREATE TABLE" command in SQL and defined all the attributes and constraints in the *Ballot* schema from Phase C. The primary key of this relation is *ballot_id*, which I declared as INT AUTO_INCREMENT. I decided to make the primary key an integer for simplicity, uniqueness, and easy indexing. Also, the AUTO_INCREMENT makes insertions easier for the user. Moreover, the attribute *voter_choice* is declared as ENUM('YES', 'NO', 'ABSTAIN') DEFAULT NULL to adhere to the requirements, "The set of possible choices as answers to each poll question is "YES", "NO", "ABSTAIN", or NULL." Also, *cast_datetime* has a datetime data type because it holds a datetime value. This attribute adheres to the requirements, "When a folk casts a ballot for the poll, the Elections Department maintains…the datetime and center where they casted the vote." Additionally, I created this relation last because it has four foreign key constraints to many previous entities, one to Voting_Registration(registration_id), one to Poll(poll_name_id), one to Folks(folk_id), and one to Voting_Center(place_id). All of these foreign keys are needed in this relation to adhere to the project description. All of these foreign keys have the action ON DELETE RESTRICT because the *Ballot* relation holds the core election results. The deletion of one of these tuples that a *Ballot* tuple depends on will invalidate an already submitted ballot, thus restriction is needed. Moreover, the unique constraint (*folk_id, poll_name_id*) enforces the requirement, "A folk can cast at most one ballot for any single poll." Lastly, the attribute *ballot_status* indicates if the ballot has been counted in the poll, is waiting to be counted, or it's been spoiled due to an error. The attribute has a not null constraint because every ballot needs this attribute.

check_voting_center_insert (Trigger 1) and check_residence_insert (Trigger 2):
These two triggers are needed to represent the disjoint relationship between *Place* and *Voting_Center* and *Residence*, where a place can be a voting center or residence, but not both. Normal SQL constraints cannot enforce this disjoint relationship between the superclass and subclass relations. These triggers enforce the disjoint relationship by checking each insert on the *Voting_Center* and *Residence* relations. Trigger 1 checks if the *place_id* to be inserted in the *Voting_Center* relation is already in the *Residence* relation. It does this by searching through each tuple of the *Residence* table and counting the amount of times the *place_id* to be inserted is found. If it's more than 0, it sets the *place_id* of the tuple to be inserted to NULL, in order to purposefully break the primary key not null constraint of *Voting_Center*. This successfully blocks the insertion. Trigger 2 does the same for an attempted insert into the *Residence* relation.

check_poll_modification (Trigger 3):
This trigger is needed to adhere to the requirement, "A poll may be modified only if there are not any cast ballots (votes) for the poll." Normal SQL constraints cannot enforce blocking an update of a poll based on the existence of a poll foreign key in the *Ballot* table. This trigger enforces this rule by searching through all of the

tuples of the *Ballot* relation, and counting the number of instances that the *poll_name_id* that is attempted to be modified exists. If the number of instances is more than 0, that means ballots have already been cast for this poll. This means that the modification of the poll tuple should be blocked. The trigger does this by setting the *poll_name_id* to null, which breaks the primary key constraint of *Poll*. This creates an intentional error and blocks the modification.

validate_registration_status (Trigger 4):
This trigger is needed to adhere to the requirement, "A folk can register to vote at a center on a date only if that date is contained in an availability period of that center; such registrations are deemed valid (registrations those that don't satisfy this condition are deemed invalid)." Normal SQL constraints cannot enforce the rules described in the project requirement and set the value of *validation_status* based on data in another relation. Thus I created this trigger to check a tuple that is being inserted into *Voting_Registration*, which checks if the *desired_voting_date* is within one of the operating periods for the designated voting center in the registration. It does this by searching through the *Operating_Period* relation for the designated voting center's availability periods, and then trying to match the dates of the operating periods to the *desired_voting_date* in the registration. If there are more than 0 matches, then the *validation_status* of the registration is set to "Valid". If not, it is set to "Invalid". This BEFORE INSERT trigger successfully validates and invalidates registrations, adhering to the logic described in the project requirements.

validate_and_fill_ballot_insert (Trigger 5):
This trigger has many functions that are needed to ensure the data integrity of a to-be-inserted tuple in the *Ballot* entity. Firstly, this trigger sets the *folk_id* and *center_id* of the *Ballot* tuple to be the same ones in the associated *Voting_Registration* tuple. This ensures that the voter and center information on the ballot is the same as the one in its connected registration. Moreover, this trigger is needed to resolve the requirements, "Attempts to vote...without a currently valid registration are rejected" and "A folk can cast a ballot...on the date and center of one of their valid registrations, and during the operating hours of that center on that date." Before a tuple is inserted into *Ballot*, this triggers checks if the associated *Voting_Registration* is valid. If not, it blocks the insertion. Moreover, this trigger checks if the *desired_voting_date* in the registration is the same as the attempted casting date of the ballot. If not, it blocks the insertion. Lastly, this trigger checks if the *cast_datetime* of the ballot is within the operating period of the voting center. If not, it blocks the insertion. Furthermore, these insertions are blocked by setting the *registration_id* of the to-be-inserted *Ballot* tuple to null, which purposefully breaks the primary key not null constraint. This successfully blocks the insertion and makes sure that an inserted tuple in the *Ballot* relation adheres to all of project requirements.

check_operating_period_modification (Trigger 6):
This trigger is needed to adhere to the requirement, "Changes to a center's operating period(s) should be allowed only if nobody has registered to vote at that center during that time period." Normal SQL constraints cannot enforce blocking an update of an operating period based on the amount of registrations that period has. This trigger enforces this rule by looking through the *Voting_Registration* relation and counting the amount of registrations that are tied to that center and operating period. If the count is more than 0, that means the registrations exist for that center's operating period. Thus, the modification is blocked by setting the new *op_id* to null, which purposefully breaks the primary key constraint of *Operating_Period*. This successfully blocks the modification, adhering to the project requirements.

check_registration_invalidation (Trigger 7):

This trigger is needed to adhere to the requirement, "Changes to polls or centers that may affect the integrity of a casted ballot should be rejected (e.g., invalidating of a cast ballot's certified valid voting registration…)." Normal SQL constraints cannot enforce blocking an update of a *Voting_Registration* that is linked to a ballot that has already been cast. This trigger enforces this rule by checking an attempted modification of a tuple in *Voting_Registration*. If the *validation_status* of the registration tuple is changing from a "Valid" state to an "Invalid" state, the trigger checks through all of the tuples in the *Ballot* table and counts the amount of ballots linked to the registration that is being modified. If any ballots exist, the trigger will block the modification by purposefully setting the *registration_id* of the modification to be null, which will violate the primary key constraint. This will block the modification successfully, adhering to the project requirements.

Justification: dropAll.sql

The purpose of the dropAll.sql file is to reset the database environment by deleting all of the triggers and relations that are made in the createAll.sql file. To reset the environment, the dropAll.sql file first drops all of the triggers using the SQL command "DROP TRIGGER IF EXISTS [trigger_name];". I decided to drop all of the triggers first before the relations to avoid any issues, as the triggers rely on and reference the relations. Next, I drop all of the relations using "DROP TABLE IF EXISTS [relation_name];". The relations are dropped in the reverse order that they are created in via the createAll.sql file. This ensures that the tables are dropped in the reverse order of their dependencies, so that none of the foreign key reference constraints prevent the deletion of any relations. As a result, referential integrity is kept throughout the deletion process without any issues.

Justification: loadAll.sql

The purpose of the loadAll.sql file is to populate the database with valid tuples that I created. To populate the database, I used the SQL command "INSERT INTO [relation_name] VALUES(...)". This file demonstrates the functionality of the database through data insertion. Moreover, I populated each relation that was created in the createAll.sql file with at least the minimum amount of data that is specified in the project requirements. I inserted 8 tuples for *Elections_Staff*, 15 tuples for *Folks*, 7 tuples for *Place* that are distributed across 3 cities and 2 states, 3 tuples for *Voting_Center*, 4 tuples in *Operating_Period* for each *Voting_Center*, 4 tuples for *Poll*, 30 tuples in *Voting_Registration*, 22 tuples in *Ballot* that is distributed across 4 polls, and 8 *Elections_Staff* that each have 4 shifts. Moreover, registrations span across 4 polls, 3 centers, and 2 months. Thus, my loadAll.sql file successfully loads the database with data adhering to the requirements.

Phase E Report:

This consists of the following:

• Develop (an) appropriate applications and/or user interface(s), using Python Jupyter Notebooks, which satisfy all functional user requirements.

• SQL scripts for creating indices for the database application. Justify the reasons for creating any such indices. Your notebooks should have appropriate cells acquiring input from the user and providing output to the user (as needed).

Justification: Creation of Indices

Place_city_index:

I created an index called Place_city_index, which is an index on the relation *Place* on the attribute *city*. I decided to make this index to optimize finding all locations (voting centers and residences) in a specific city. I believe this index is needed, as queries that involve the gathering of election information specific to a particular city will be common within this database. It is incredibly common and necessary to check how a particular city votes in elections. Moreover, the attribute *city* is a non-unique attribute, thus without an index, queries using the *city* attribute would require a full scan of *Place*, which would be highly inefficient. Lastly, this index is particularly useful for the Query/Report B.6, which requires finding the most popular voting centers in a specific city. Therefore, this index is necessary.

Voting_Registration_center_date_index:
I created an index called Voting_Registration_center_date_index, which is an index on the relation *Voting_Registration* on the attributes *center_id* and *desired_voting_date*. I decided to make this index to optimize finding voting registrations tied to a specific voting center and/or range of dates. I believe this index is needed, as queries that involve the gathering of voting registration information specific to a particular voting center and/or time period will be common within this database. It is very common and necessary to check and view a report for how many people register to vote at a specific voting center and what time period they will be voting. Furthermore, without this index, queries looking for information regarding a voting center within a certain date range would require a full scan of the *Voting_Registration* relation, which is very resource intensive. This composite index allows the database to easily find information regarding a voting center and the given range of dates. Lastly, this index is particularly useful for the Queries/Reports B.4 and B.6, which require querying information for a voting center within a given time period.

Place_state_index:
I created an index called Place_state_index, which is an index on the relation *Place* on the attribute *state*. I decided to make this index to optimize finding all locations (voting centers and residences) in a specific state. I believe this index is needed, as queries that involve the gathering of election information specific to a particular state will be common within this database. It is incredibly common and necessary to check how a particular state votes in elections. Moreover, the attribute *state* is a non-unique attribute, thus without an index, queries using the *state* attribute would require a full scan of *Place*, which would be highly inefficient. Lastly, this index is particularly useful for the Query/Report B.7, which requires finding folks registered at every center in a given state. Therefore, this index is necessary, as Query/Report B.7 is a complex query involving isolation of voting centers in a specified state.

Justification: transaction.sql Isolation Level

For the file transaction.sql, which is made for Activity A.4, I decided to set the isolation level to the strictest setting of SERIALIZABLE. I defined the isolation level to be SERIALIZABLE for the activity, "a voter casting a ballot while confirming a valid voting registration," because it ensures data integrity and consistency, and offers the highest protection against dirty reads, non-repeatable reads, and phantom reads. This particular transaction deals with the highly sensitive activity of voters casting their ballots for the election. Thus, it is imperative that this transaction guarantees that no phantom reads take place. This is to prevent double-voting, as by locking the data, the database avoids the situation where two simultaneous transactions read that a folk has zero ballots for a certain poll, then simultaneously inserts two ballots. SERIALIZABLE makes sure that the data is locked as soon as one transaction begins, thus preventing double-voting. Averting phantom reads cannot be done with any of the other three isolation levels, therefore the best isolation level for this transaction is SERIALIZABLE. Lastly, it is important that the elections staff do not have any dirty reads and non-repeatable

reads when querying the database, as that can result in inaccurate election results. Overall, the top choice is the SERIALIZABLE isolation level for this activity.

Phase F Report:

The user guide can be seen at the top of the Jupyter Notebook file.